



BTGenBot × NAV4RAIL

Génération de Behavior Trees avec LLMs compacts

Benjamin Lepourtois Anne Faury Mohamed Amar
Mourad Latoundji

19 mai 2026

Mastère spécialisé IA/DATA — Promotion 2026



Contexte & Problème

BTGenBot — Article de référence

Lien avec la SNCF

Plan de reproduction

Conclusion

Ce que veut la SNCF :

- Décrire une mission en langage naturel
- Obtenir un **template BT paramétré** en XML
- Template instancié par un fichier de config au runtime
- Nœuds = catalogue fermé de **28 skills** NAV4RAIL

Contrainte principale :

Un seul exemple réel disponible à ce jour

Template SNCF (cible)

```
<Sequence>
  <LoadMission
    mission_file_path=
      "{mission_file_path}"/>
  <MissionStructureValid/>
  <GenerateInspectionPath
    origin_sph="{origin_sph}"
    dest_sph="{dest_sph}"/>
  <Fallback>
    <
      NavigateToInspectionPoint
    />
    <HandleFailure/>
  </Fallback>
</Sequence>
```

Pourquoi on ne peut pas générer le dataset proxy ?

Tentative dataset proxy

- Génération synthétique par LLM
- Résultat : **très loin des BTs réels**
- Structure des BTs incorrecte, câblage des ports erroné
- **Aucune connaissance métier ferroviaire**

Solution retenue

Reproduire un article de référence sur **données publiques réelles**, puis argumenter le transfert SNCF.

Ce qu'il faudrait idéalement :

- 20–50 templates BT réels SNCF
- Descriptions associées en NL
- Fichier de config avec variables blackboard

Ce qu'on a :

- 1 exemple réel (mission d'inspection)
- Catalogue de 28 skills avec ports
- 27 règles de sécurité (L1–L5)

Contexte & Problème

BTGenBot — Article de référence

Lien avec la SNCF

Plan de reproduction

Conclusion

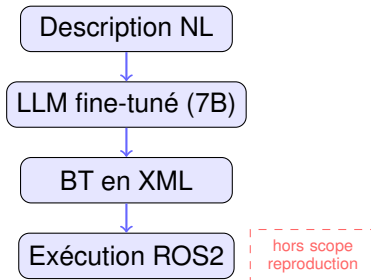
Hypothèse centrale :

Un LLM compact (7B paramètres) fine-tuné sur **600 exemples réels** génère des BTs de qualité comparable aux grands modèles.

4 contributions :

1. Dataset 600 paires BT ↔ description NL
2. Système d'évaluation sur 9 tâches
3. Comparaison 3 LLMs × base vs fine-tuné
4. Pipeline exécution sur robot physique

Pipeline BTGenBot :



Construction du dataset :

1. **600 BTs réels** extraits de projets ROS2 open-source (Nav2, manipulation, exploration) — validés sur robots
2. **GPT-3.5-turbo** génère la description NL de chaque BT (prompt one-shot, max 200 mots)
3. Format **Alpaca** : `instruction / input / output`

Fine-tuning LoRA :

Paramètre	Valeur
Modèles	LLaMA-2-7b / Chat / CodeLlama
Méthode	LoRA ($r=8$, $\alpha=16$)
Modules	q/k/v/o_proj + MLP (7)
Epochs	70–80
LR	3e-4
Contexte	2048 tokens
GPU	2× RTX Quadro 6000

Baseline zero-shot

(grands modèles, sans FT)

Modèle	Synt.	Sém.
GPT-3.5 (175B+)	100%	77.8%
Gemini (~1T)	88.9%	55.6%
LLaMA-2-13b	33.3%	22.2%
LLaMA-2-7b	0%	—

Les 7B sans FT sont inutilisables.

Après fine-tuning (ZS)

Phase 1 — tâches 1–7

Modèle	Synt.	Sém.	Config.	BT OK
LLaMAChat-7b FT	71.4%	57.1%	LLaMAChat-7b OS+SA	77.7%
CodeLlama-7b FT	85.7%	100%	CodeLlama-7b OS+SA	33.3%

OS = one-shot SA = static analysis

Validation complète

Phase 2 — one-shot + correction

Nouvelle baseline à évaluer : Claude Opus 4.7 (Anthropic, mai 2026) en zero-shot sur les 9 tâches — quelle est la limite des grands modèles sans fine-tuning en 2026 ?

Contexte & Problème

BTGenBot — Article de référence

Lien avec la SNCF

Plan de reproduction

Conclusion

Ce que BTGenBot prouve

- 600 exemples + LoRA $\Rightarrow$ BTs valides
- Domaine ouvert (ROS2 varié)
- Nœuds libres, ports non typés
- BTs statiques (pas de templates)

SNCF : domaine plus contraint

- **Vocabulaire fermé** : 28 skills fixes
- **Ports typés** par le catalogue
- **Variables blackboard** :
`{mission_fp}`
- **Templates** instanciés par config

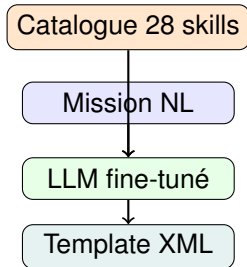
Argument central

Domaine contraint = espace de génération réduit = **moins d'exemples nécessaires.**

20–50 templates réels SNCF suffisent pour reproduire les performances de BTGenBot.

Différence architecturale avec BTGenBot :

Le catalogue est injecté **dans le prompt** à chaque inférence :



Ce que le LLM apprend avec 20–50 exemples :

- Sélectionner les skills pertinents parmi 28
- Câbler les ports avec les bonnes {variables}
- Respecter les prérequis (LoadMission avant Valid)
- Produire un template réutilisable

Le catalogue agit comme **prior structural fort** : le LLM choisit parmi 28 options, il n'invente plus.

Contexte & Problème

BTGenBot — Article de référence

Lien avec la SNCF

Plan de reproduction

Conclusion

Reproduit directement :

Composant	Source
Dataset	600 paires (GitHub public)
Fine-tuning LoRA	QLoRA, modèles 2024
Éval. syntaxique	Valideur XML/BT
Éval. sémantique	TED + Node-F1 + LLM-judge
Base vs FT	Tableau 9 catégories

Adapté / non reproduit :

Composant	Raison
Simulation ROS2	Non disponible
Robot physique	Hors scope
LLaMA-2 original	→ LLaMA-3.1/Qwen2.5

Évaluation sémantique sans ROS2

- **TED** : nb d'opérations pour transformer le BT généré en BT référence
- **Node-F1** : quels nœuds du BT référence sont présents dans la génération
- **LLM-judge** : Phi-3-mini dit si le BT correspond à la description NL (77.8% accord GPT-4, validé dans l'article)

Semaine 1 — Appropriation du dataset

- Téléchargement dataset BTGenBot (GitHub)
- **EDA** : catégories, longueur BTs, vocabulaire nœuds
- Conversion JSONL + adaptation prompt

Semaine 2 — Fine-tuning

- SFT QLoRA sur 3 modèles (cluster Télécom, RTX 3090)
- LLaMA-3.1-8B, Qwen2.5-7B, Qwen2.5-Coder-7B
- ~2–4h par modèle

Semaine 3 — Évaluation

- Correction syntaxique (validateur BT)
- TED + Node-F1 vs référence
- LLM-judge sur 9 catégories
- Zero-shot vs one-shot

Semaine 4 — Analyse & Rédaction

- Tableau comparatif base vs FT
- Discussion transfert SNCF
- Rédaction rapport final

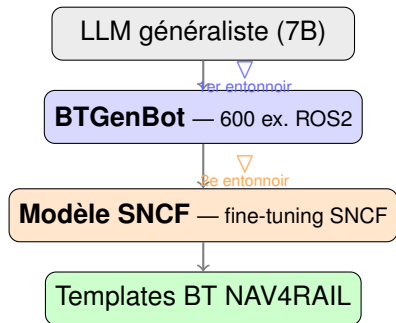
Contexte & Problème

BTGenBot — Article de référence

Lien avec la SNCF

Plan de reproduction

Conclusion



Ce que BTGenBot apporte :

- Prouve qu'un LLM 7B peut apprendre la **syntaxe et la sémantique des BTs**
- Fournit un modèle **déjà spécialisé** — pas un LLM généraliste en point de départ
- Réduit l'effort du second fine-tuning SNCF

Message clé

BTGenBot = **1^{er} entonnoir** (domaine BT). Le **2^e** entonnoir (domaine SNCF) reste à construire dès que la SNCF fournit ses exemples réels.

Paramètre	BTGenBot original	Notre reproduction
Modèles	LLaMA-2-7b, Chat, CodeLlama	LLaMA-3.1-8B, Qwen2.5-7B/Coder
PEFT	LoRA	QLoRA (NF4)
lora_r	8	8–16
lora_alpha	16	16–32
Modules cibles	7 (q/k/v/o + MLP)	identique
Epochs	70–80	70–80
Learning rate	3e-4	2e-4–3e-4
Dataset	600 paires	600 paires (identique)
GPU	2× RTX Quadro 6000	RTX 3090 (cluster Télécom)

1. **Navigation** : visiter des positions dans l'ordre
2. **Navigation avec priorité** : seuil sur lecture capteur
3. **Navigation avec fallback** : sauter waypoints inaccessibles
4. **Navigation + bras** : activer manipulateur à chaque position
5. **Exploration** : navigation continue, check complétion
6. **Exploration manipulateur** : cycler configs articulaires
7. **Vision active + saisie** : observer, estimer prise, pick-and-place
8. **Traitement matériaux** : séquence de boutons + évaluation statut
9. **Assemblage multi-station** : collecter composants, assembler

Phase 1 (sélection) = tâches 1–7 Phase 2 (validation) = tâches 1–9